Linked List & Hashing Practice Questions

March 30, 2025

1 Linked List

1. Create a `struct` representing each node of a Doubly Linked List. It should contain an integer value and pointers to the previous and next node.

2. Create a function called `printList()` which takes a pointer to the head node and prints the entire list. The format should be the value of each node followed by '->' to show connections. You can use the following code:

```
int main() {
    // Create a linked list with 4 nodes
    struct ListNode a, b, c, d;
    a.val = 1; a.next = &b;
    b.val = 2; b.next = &c;
    c.val = 3; c.next = &d;
    d.val = 4; d.next = NULL;

    // *Implement this*
    printList(&a);
    return 0;
}
```

   The output should be:

```
1 -> 2 -> 3 -> 4 -> NULL
```

3. Create a function called `swap()` which should swap the first two nodes in the list, preserving all other connections. It should return a pointer to the new head of the list. (Assume size of the list is >=2). For example:
   Input: 1 –> 2 –> 3 –> 4 –> NULL
   Output: **2** –> **1** –> 3 –> 4 –> NULL

4. There are several Linked List questions described in the textbook (ex. reverse a list, return Nth node, ...), so you should review those too.

2 Hashing

1. On average, What is the runtime complexity of searching for an element in your HashMap (i.e. in big O notation)? Explain why, using the steps that the system takes to retrieve this element.

2. Describe what a collision means in a HashMap, and what is one way you can handle collisions when they occur?

3. Complete the HashMap `insert()` function below, filling in the code under the comments. What is the average runtime complexity of inserting an element into a HashMap? What can cause performance to be reduced?

```c
#include <stdio.h>
#include <stdlib.h>

#define TABLE_SIZE 10

typedef struct {
    int key;
    int value;
} HashEntry;

HashEntry *hashTable[TABLE_SIZE] = {NULL};

// Simple hash function
int hashFunction(int key) {
    return key % TABLE_SIZE;
}

void insert(int key, int value) {
    // 1) hash the key
    // ...

    // 2) create a HashMap entry with appropriate key and
       value
    // ...

    // 3) store the entry in the HashMap
    // ...
}
```